Quantum Random Number Generator

Day 21 | 30-Day Quantum Computing Challenge — Mini Project

Quick recap. Day 19 prepared and measured a balanced qubit. Day 20 compared repeated measurements. Here we use the same 50/50 measurement idea to construct a longer binary sequence and examine its properties.

1. Objective

Generate a configurable number of bits (default: 100), print the sequence, count zeros and ones, and visualize their frequencies and running proportion. This project extends the earlier Quantum Randomness Laboratory concept as a compact version-2 exercise.

2. Quantum Principle

Prepare |0>, apply a Hadamard gate, and measure. The ideal output probabilities are P(0) = |1/sqrt(2)|² = 1/2 and P(1) = 1/2. Repeat the experiment to obtain multiple bits. Each shot requires a fresh preparation and measurement.

```
|0> ■■ H ■■ Measure ■■> 0 or 1
```

3. Important Simulation Distinction

The notebook runs immediately using seeded NumPy sampling of the **ideal quantum measurement distribution**. These bits are pseudo-random simulation outputs, **not certified physical quantum randomness**. An optional Qiskit Aer circuit simulates the actual gate-and-measurement workflow; Aer is also a simulator, not quantum hardware. A physical QRNG would need hardware, calibration, and entropy assessment.

4. Experiment Procedure

(1) Set **number_of_bits = 100**. (2) Generate independent binary outcomes. (3) Display the bit string and frequency table. (4) Plot counts and the running proportion of ones. (5) Try 10, 100, 1,000 bits and compare fluctuations. (6) Optionally run the Qiskit Aer version.

5. Example Output

Seed: 2026 **Bits:** 100 **Zeros:** 53 (53.0%) **Ones:** 47 (47.0%). The counts need not be exactly equal in a finite sequence.

```
First 40 bits: 1001000010111110110001110111011110000000
```

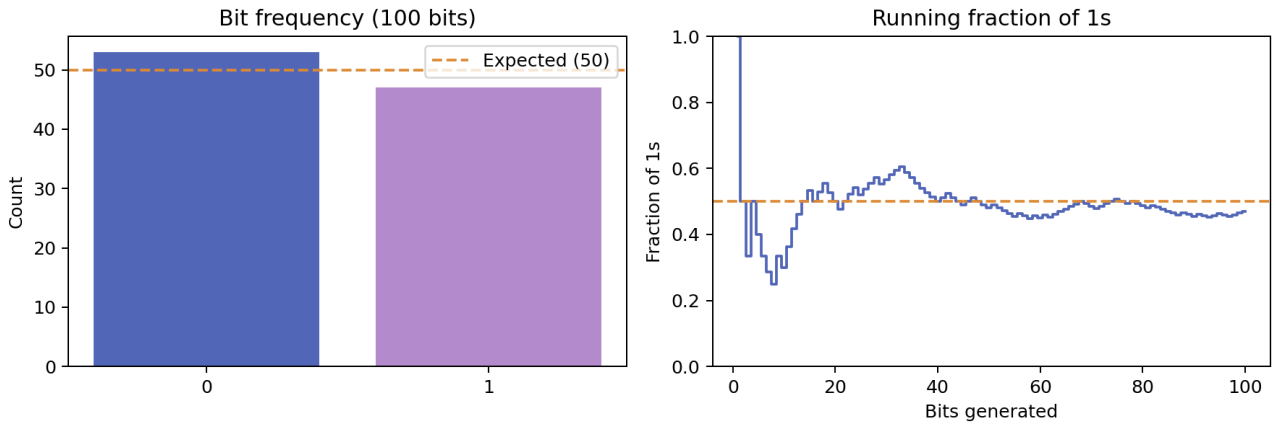


Figure 1. Frequency distribution and running proportion for the seeded 100-bit experiment.

6. Interpreting the Results

A near-50/50 frequency is consistent with a balanced source, but it does not prove unpredictability, independence, cryptographic security, or quantum origin. A histogram is only a basic sanity check; serious QRNG validation requires additional statistical and physical-source analysis.

7. Run and Reproduce

Install: `pip install numpy matplotlib jupyter`. Open `quantum_random_number_generator.ipynb` and select Run All. Optional Qiskit section: `pip install qiskit qiskit-aer`. The notebook regenerates `results.png`.

8. Summary: What to Remember

A Hadamard-plus-measurement experiment produces ideal balanced binary outcomes. Repetition creates a bit string; sample counts fluctuate. Simulation is useful for learning but must not be presented as a hardware-certified quantum random-number generator.